Recommender at Scale: Latency, Cost and Popularity Bias

Comprehensive Teammate Learning, Architecture Walkthrough & Viva Preparation Guide
Course: Big Data Systems (Final Demo: 2 November 2026)
Team: Sripada Krishna Sameer Aditya (PES1UG24AM285), Shree Verdhan M (PES1UG24AM265), Shrivamshi Chikkireddi (PES1UG24AM287)
Repository: https://github.com/krishna28-sam/Recommender-at-Scale-Latency-Cost-and-Popularity-Bias

[HOW TO USE THIS GUIDE] This guide is designed as an end-to-end, self-contained textbook and examination prep manual for our project. Every team member can master the entire system, present each slide confidently, and answer examiner questions accurately. All figures, latencies, CPU metrics, and fairness statistics are loaded directly from our verified benchmark outputs.

Table of Contents

- Chapter 1: Project Overview & Executive Summary
- Chapter 2: Fundamentals of Big Data & Stream Processing
- Chapter 3: End-to-End System Architecture & Data Flow
- Chapter 4: Dataset, Preprocessing & Privacy Engineering
- Chapter 5: Recommendation Methodology & Evaluation Metrics
- Chapter 6: Popularity Bias, Exposure Inequality & Fair Re-Ranking
- Chapter 7: Source Code & Module Walkthrough
- Chapter 8: Experimental Setup, Capacity Probe & Benchmark Matrix
- Chapter 9: Results, Analysis & Engineering Insights
- Chapter 10: Limitations, Methodological Caveats & Future Work
- Chapter 11: Installation, Execution & Reproducibility Guide
- Chapter 12: Team Presentation Strategy & Role Division
- Chapter 13: Comprehensive Viva Voce Question Bank (30 Questions)
- Chapter 14: Quick Revision Sheets & Numerical Cheat Sheet

Chapter 1: Project Overview & Executive Summary

Project Title: Full Title: Recommender at Scale: Latency, Cost and Popularity Bias in Streaming News Recommendation.

Abstract: This project presents an empirical, large-scale systems comparison between two premier open-source distributed stream processing engines—Apache Spark Structured Streaming and Apache Flink (PyFlink)—applied to real-time news recommendation using the real Microsoft News Dataset (MIND-small). We evaluate end-to-end event latency, throughput capacity, CPU efficiency, memory footprints, and cloud hosting costs under varying ingestion rates (100 to 4,000 events/second). Concurrently, we investigate and mitigate popularity bias and publisher exposure inequality in streaming recommendations via an in-stream, exposure-aware greedy re-ranking algorithm. Our findings establish the operational limits, buffering tradeoffs, and latency-fairness boundaries of real-time recommender systems.

Problem Statement & Real-World Motivation

Modern digital platforms (such as news aggregators, social feeds, and e-commerce stores) serve millions of active users concurrently. Recommender systems on these platforms face two simultaneous, conflicting challenges: (1) Systems-level challenge: Ingestion rates fluctuate wildly, yet recommendations must be served within sub-second latencies before users navigate away. Processing streaming updates with micro-batching or event-driven execution creates significant tradeoffs between latency, throughput, and compute

cost. (2) Algorithmic fairness challenge: Standard click-through rate (CTR) prediction models suffer from severe popularity bias—a vicious feedback loop where popular articles from dominant publishers receive exponentially more impressions, starving smaller or emerging publishers of visibility even when their content is high quality.

What Makes This a Big Data Systems Project?

This is not merely a machine learning project; it is fundamentally an engineering study of distributed data systems under heavy ingestion. It involves: (1) Multi-partition distributed message brokering with Apache Kafka; (2) Dual streaming engine paradigms: micro-batching (Spark Structured Streaming) versus low-latency pipelined event-driven streaming (Apache Flink); (3) Real-time state management across continuous event streams; (4) Strict saturation profiling (identifying the exact breaking point where input rate exceeds engine processing capacity); (5) Cryptographic user pseudonymization meeting enterprise privacy standards.

[THE 2-MINUTE ELEVATOR PITCH FOR EXAMINERS] Our project compares Apache Spark Structured Streaming against Apache Flink on the real MIND-small dataset under streaming ingestion rates up to 4,000 events/second. We measure latency, CPU cost, and queue backlog across a 42-run benchmark matrix, showing where single-node streaming engines saturate (~1,730 events/sec) and how Flink's bundle buffering parameter drops latency from 1,008 ms to 216 ms. At the same time, we solve the popularity bias problem by implementing a real-time fair re-ranking algorithm that slashes publisher exposure inequality by 40% with only a 1.2% to 4.4% utility drop.

Chapter 2: Fundamentals of Big Data & Stream Processing

To explain our project clearly, you must first master the fundamental concepts governing modern distributed data systems.

Concept Breakdown: Batch vs. Stream Processing

- **1. What it means in simple language:** Batch processing processes large volumes of static, historical data all at once (e.g., overnight MapReduce or daily SQL reports). Stream processing processes data continuously as individual events or micro-batches arrive in real time with minimal latency.
- **2. Why it is needed in our project:** News recommendations are highly perishable; an article published 10 minutes ago loses relevancy if recommendations are updated only in daily batches. Stream processing allows instant personalization.
- **3. How we implemented it:** We implemented streaming consumers in both PySpark (`spark/pass_through.py`) and PyFlink (`flink/pass_through.py`) subscribing to live Kafka impression streams.
- **4. Where it appears in the source code:** `spark/pass_through.py` (lines 44-58), `flink/pass_through.py` (lines 36-52).
- **5. How to explain it to an examiner:** We use batch processing offline to precompute static CTR smoothing tables and user category histories, but use stream processing online to ingest live user impressions and emit top-5 recommendations in real time.
- **6. Possible follow-up question & answer:** Q: Why not update user histories in real time too? A: In our setup, static tables represent pre-indexed candidate pools, keeping per-event streaming latency minimal (<300 ms).

Concept Breakdown: Apache Kafka & Message Brokering

- **1. What it means in simple language:** Kafka is a distributed, append-only, partitioned commit log. Producers append events to topics, and consumers read events sequentially by tracking their offset (position).
- **2. Why it is needed in our project:** Kafka acts as the high-throughput, fault-tolerant transport buffer between our replay producer (which generates user clicks) and the streaming recommendation engines.
- **3. How we implemented it:** Docker Compose launches Kafka with two topics: `impressions` (input) and `recommendations` (output), configured with 2 partitions and `LogAppendTime`.

- **4. Where it appears in the source code:** `docker-compose.yml` (kafka service), `Makefile` (`topics:` rule lines 45-48), `producer/replay.py`.
- **5. How to explain it to an examiner:** Kafka decouples ingestion rate from engine processing speed. By using `LogAppendTime`, Kafka timestamps each message when it hits the broker disk, enabling hardware-accurate end-to-end latency measurements without clock skew.
- **6. Possible follow-up question & answer:** Q: What is the significance of 2 partitions? A: Partitions enable parallel consumption. However, if state is kept in engine memory, having 2 partitions can split the re-ranking state across workers.

Concept Breakdown: Spark Structured Streaming vs. Apache Flink

- **1. What it means in simple language:** Spark Structured Streaming is a micro-batch engine: it accumulates incoming events over a small time window (trigger) and processes them as an RDD/DataFrame. Apache Flink is a true event-driven engine: each record flows continuously through an operator pipeline one by one.
- **2. Why it is needed in our project:** Comparing these two dominant architectures is the core research question of our project.
- **3. How we implemented it:** In Spark, we configured `.writeStream.foreachBatch(process_batch)` with either eager default trigger or 1-second fixed trigger. In Flink, we used `StreamExecutionEnvironment` with KafkaSource and `map(process)` sink.
- **4. Where it appears in the source code:** `spark/pass_through.py` vs. `flink/pass_through.py`.
- **5. How to explain it to an examiner:** Spark's micro-batching amortizes driver overhead and reduces CPU consumption at high load, but adds latency. Flink's event-driven pipeline offers lower per-event latency, but requires tuning Python worker bundle buffers to avoid buffering delays.
- **6. Possible follow-up question & answer:** Q: Why did PyFlink have a 1,008 ms latency out-of-the-box? A: PyFlink uses an internal bundle buffer (`python.fn-execution.bundle.time`) which defaults to 1,000 ms before flushing to the Python worker!

Concept Breakdown: Latency vs. Throughput & Saturation

- **1. What it means in simple language:** Latency is the time taken to process a single event from arrival to output (measured in milliseconds). Throughput is the number of events processed per unit time (events per second). Saturation occurs when arrival rate exceeds maximum throughput, causing queues to grow indefinitely.
- **2. Why it is needed in our project:** We must determine whether low-latency engines can sustain enterprise traffic without collapsing under backpressure.
- **3. How we implemented it:** We measured Kafka `LogAppendTime` latency differences and tracked backlog at the end of 60-second measurement windows across rates from 100 to 4,000 ev/s.
- **4. Where it appears in the source code:** `bench/run.py` (lines 350-420), `bench/probe.py`.
- **5. How to explain it to an examiner:** A system can have excellent p50 latency (200 ms) at 1,000 ev/s, but if pushed to 2,000 ev/s when its maximum capacity is 1,730 ev/s, input events pile up in Kafka, causing p99 latency to explode to 13,000 ms and backlog to exceed 18,000 events.
- **6. Possible follow-up question & answer:** Q: How do you detect saturation? A: When output throughput plateaus while input rate increases, and queue lag slope becomes positive.

Chapter 3: System Architecture & Data Flow

Our project architecture consists of five core stages: Ingestion, Transport, Streaming Inference, Output Collection, and Benchmark Accounting.

Architecture Diagram & Component Descriptions

Component	Source Files	Primary Function	Key Setting
Replay Producer	producer/replay.py, docker/producer.Dockerfile	Reads behaviors.tsv, hashes user IDs with salted SHA-256, streams JSON to Kafka	Poisson/uniform rate pacing, MIND_USER_HASH_SALT
Kafka Broker	docker-compose.yml	High-throughput partitioned log buffer storing input and output events	2 partitions, LogAppendTime, 9092/29092

Spark Streaming	spark/pass_through.py, docker/spark.Dockerfile	Micro-batch consumer, loads model tables, computes Laplace CTR + fair re-ranking	PROCESSING_TIME triggers: default vs 1s
Flink Streaming	flink/pass_through.py, docker/flink.Dockerfile	Continuous DataStream consumer, event-by-event scoring and fair re-ranking	python.fn-execution.bundle.time (default vs 50ms)
Benchmark Monitor	bench/run.py, bench/probe.py	Measures consumer offsets, log timestamps, drain duration, and cgroup CPU/memory	60s window, 3 reps, drain timeout

Step-by-Step Lifecycle of a News Impression Event

Follow this exact step-by-step narrative if an examiner asks: 'Walk me through an event from start to finish':

- Step 1 (Replay & Anonymization): The replay producer reads a user impression row containing `user_id`, `history` (previously clicked news IDs), and `candidates` (candidate news IDs displayed to the user). It cryptographically hashes the user ID using SHA-256 and an environment salt (`MIND_USER_HASH_SALT`), ensuring zero raw user IDs enter Kafka.
- Step 2 (Ingestion into Kafka): The producer serializes the impression into a JSON payload and produces it to Kafka topic `impressions`. The Kafka broker timestamps the record with its exact disk write time (`LogAppendTime`).
- Step 3 (Streaming Engine Ingestion): Spark or Flink consumes the record. Spark pulls a micro-batch of records; Flink pulls records via its KafkaSource DataStream.
- Step 4 (Scoring & Ranking): For each candidate news item in the impression, the engine queries precomputed Laplace-smoothed CTR and category affinity scores. Candidate items are sorted by CTR in descending order.
- Step 5 (Fairness-Aware Re-Ranking): The top candidates enter the fair re-ranker. The re-ranker maintains a running tally of publisher exposure. If a publisher is over-represented relative to its catalog target, its candidates receive a penalty proportional to parameter lambda. The top-5 fairest items are selected.
- Step 6 (Output to Kafka): The served top-5 recommendation list is serialized to JSON and emitted to topic `recommendations`. Kafka assigns a second `LogAppendTime` timestamp.
- Step 7 (Benchmark Measurement): The benchmark harness inspects the input and output Kafka log timestamps, computes the exact difference (end-to-end latency), verifies event ID parity, and records container cgroup CPU and memory stats.

Chapter 4: Dataset, Preprocessing & Privacy Engineering

Our research relies on the real Microsoft News Dataset (MIND-small).

MIND-small Dataset Characteristics

MIND-small is a real-world benchmark dataset collected from anonymized user click logs on Microsoft News over a 6-week period:

- **Training Set:** MINDsmall_train: Contains 51,282 news articles and 156,965 user impression logs.
- **Validation/Dev Set:** MINDsmall_dev: Contains 42,416 news articles and 73,152 user impression logs.
- **News Features:** News Schema: `news_id`, `category`, `subcategory`, `title`, `abstract`, `url`, `title_entities`, `abstract_entities`.
- **Behaviors Features:** Behaviors Schema: `impression_id`, `user_id`, `time`, `history` (space-delimited list of past clicked news IDs), `impressions` (list of candidate news IDs with binary click labels: 1=click, 0=non-click).

Synthetic Publisher Group Mappings (Critical Honest Disclosure)

[CRITICAL DISTINCTION FOR EXAMINERS] CRITICAL VIVA POINT: MIND-small does NOT contain a publisher or news source field! To rigorously study publisher exposure inequality, we constructed six synthetic controlled publisher mappings across the union of 65,234 unique news articles. Never tell an examiner that these are real news companies like CNN or BBC. They are controlled experimental groups.

We generated six synthetic publisher configurations (`data/processed/publishers/`):

- **1. Random Uniform Family:** random_101, random_202, random_303: 100 synthetic publishers assigned uniformly at random to articles using random seeds 101, 202, and 303.
- **2. Category-Correlated Family:** category_404, category_505, category_606: 100 synthetic publishers assigned such that publishers specialize in specific news categories (e.g., sports, finance, lifestyle) using category priors and Dirichlet-multinomial sampling.

Privacy Engineering & Salted SHA-256 Hashing

To comply with strict privacy standards (GDPR/CCPA) and project requirements: Raw user IDs (e.g., `U13740`) are never transmitted over Kafka or written to benchmark logs. In `scripts/hash_users.py` and `producer/replay.py`, user IDs are hashed: `hashed_id = sha256((user_id + salt).encode('utf-8')).hexdigest()`. The salt is loaded from environment variable `MIND_USER_HASH_SALT` and is never committed to GitHub.

Chapter 5: Recommendation Methodology & Evaluation Metrics

Our recommender implements a standard point-wise scoring model enhanced with Laplace smoothing and category affinity.

Candidate Scoring Formula

For a user with click history H and candidate news item i with category c_i:

Scoring Equation: Score(i, user) = w_pop * SmoothedCTR(i) + w_cat * CategoryAffinity(c_i, H)

- **Weights:** w_pop = 0.7 (popularity weight) and w_cat = 0.3 (category affinity weight), configured in `config/model.yaml`.
- **Laplace Smoothing:** SmoothedCTR(i) = (clicks_i + alpha * prior_ctr) / (impressions_i + alpha), where alpha = 10.0 is the Laplace smoothing constant and prior_ctr = 0.05. This prevents cold-start articles with 1 click from receiving an unrealistic 100% CTR.
- **Category Affinity:** CategoryAffinity(c_i, H) = (count of category c_i in user history H) / (total clicks in H). If user history is empty, affinity defaults to the global category distribution prior.

Evaluation Metrics (Mathematical Definitions & Small Examples)

We evaluate ranking utility using standard Information Retrieval metrics:

1. nDCG@K (Normalized Discounted Cumulative Gain at Rank K)

Measures the quality of ranking by rewarding relevant items placed at the top of the recommendation list, with logarithmic position discounting:

Formula: DCG@K = sum_{r=1}^K (2^{y_r} - 1) / log2(r + 1), where y_r is the binary ground-truth click label at rank r. nDCG@K = DCG@K / IDCG@K, where IDCG@K is the ideal DCG obtained by sorting items perfectly by ground-truth label.

- **Example:** Numerical Example: Suppose at K=5, ground-truth clicks are [0, 1, 0, 0, 0] (click at rank 2). DCG@5 = (2^1 - 1) / log2(2 + 1) = 1 / log2(3) = 1 / 1.585 = 0.6309. The ideal ranking has the click at rank 1: IDCG@5 = (2^1 - 1) / log2(1 + 1) = 1 / 1.0 = 1.0. Thus, nDCG@5 = 0.6309 / 1.0 = 0.6309. If the click were at rank 1, nDCG@5 = 1.0.
- **Project Interpretation:** Candidate nDCG@10 evaluates the unconstrained top-10 candidate ranking; Served nDCG@5 evaluates the final top-5 recommendations served to the user after fair re-ranking.

2. AUC (Area Under the ROC Curve)

Measures the probability that a randomly chosen clicked article is ranked higher than a randomly chosen unclicked article. In our project, AUC is computed per impression and averaged across all 73,152 validation impressions. Impressions with only clicks or only non-clicks are assigned AUC = 0.5 (uninformative prior convention).

Chapter 6: Popularity Bias, Exposure Inequality & Fair Re-Ranking

What is Popularity Bias & Why Does It Matter?

In competitive news ecosystems, standard CTR-based recommenders concentrate user attention on a tiny handful of viral articles. In our MIND-small analysis, the top 5% of articles capture over 60% of all user clicks. When a recommender greedily serves high-CTR articles, articles from dominant publishers monopolize user screens, creating exposure inequality and starving emerging content creators.

Fairness Metric: Exposure L1 Distance & Gini Coefficient

Let p_g be the actual exposure share of publisher group g (weighted by position discount 1/log2(r+1)), and let q_g be the target catalog share of group g (proportion of articles published by group g):

Exposure L1 Formula: Exposure L1 = 0.5 * sum_{g=1}^G |p_g - q_g|

- **Interpretation:** Exposure L1 measures total variation distance between actual exposure and fair catalog representation. L1 = 0.0 means perfect proportionality; L1 = 1.0 means complete publisher monopoly.
- **Gini Metric:** Gini Coefficient: Measures exposure inequality across publishers on a scale from 0.0 (perfect equality) to 1.0 (maximal inequality).

Greedy Fair Re-Ranking Algorithm

At each step of serving top-K recommendations, the re-ranker selects the next article i that maximizes:

Objective Function: Argmax_{i in Remaining} [Score(i) - lambda * Penalty(group(i), CurrentState)]

Where Penalty(group(i), CurrentState) = max(0, (exposure_{group(i)} + 1/log2(r+1)) / (total_exp + 1/log2(r+1)) - target_{group(i)}). Parameter lambda controls the fairness penalty: lambda = 0.0 yields pure greedy CTR scoring; higher lambda values penalize over-represented publishers.

CRITICAL EXPERIMENT DISTINCTION: Lambda = 5.0 vs. Lambda = 0.6

[CRITICAL DISTINCTION: OFFLINE EVALUATION VS. STREAMING OVERHEAD] DO NOT CONFUSE THESE TWO EXPERIMENTS IN YOUR VIVA:
1. Offline Fairness/Utility Pareto Analysis: Tested lambda in [0.0, 0.1, 0.3, 0.6, 1.0, 2.0, 5.0] on 73,152 MIND-small impressions. At lambda=5.0, Exposure L1 dropped by 40% to 44% with only a 1.2% to 4.4% drop in served nDCG@5.
2. Streaming Benchmark Matrix Overhead Experiment: Tested lambda = 0.6 in live streaming runs (spark-default-rerank and flink-p1-rerank). Lambda = 0.6 added LESS THAN 1% CPU overhead and 4% latency overhead below saturation.

Chapter 7: Source Code & Implementation Walkthrough

Review this chapter to speak knowledgeably about the exact files, classes, and functions in the repository.

Module / File	Key Classes / Functions	Inputs & Outputs	Engineering Logic
recsys/scorer.py	rank_candidates(), compute_ctr()	In: candidates, user history, stats Out: scored candidate list	Computes Laplace smoothed CTR and category affinity scores
recsys/reranker.py	rerank(), exposure_penalty()	In: candidates, publisher targets, lambda Out: top-5 re-ranked items	Greedy position-discounted fair re-ranking penalty calculation
recsys/streaming.py	EventScorer, pass_through(), validate_smoke_offsets()	In: streaming JSON events Out: served recommendations	Shared stateful inference wrapper for both Spark and Flink
producer/replay.py	ReplayProducer, send_event()	In: behaviors.tsv, salt Out: Kafka impressions topic	Paces streaming ingestion; hashes user IDs with SHA-256
spark/pass_through.py	process_batch(), main()	In: Kafka impressions	PySpark Structured

		Out: Kafka recommendations	Streaming with foreachBatch and checkpointing
flink/pass_through.py	process(), main()	In: KafkaSource Out: KafkaSink	PyFlink DataStream pipeline with bundle buffer tuning
bench/run.py	run_matrix(), measure_run()	In: config, rate, rep Out: CSV latency & manifest	Orchestrates 42 benchmark runs, verifies event parity and cgroups
bias/offline_metrics.py	evaluate_predictions(), l1_distance()	In: predictions JSONL Out: bias_summary.csv	Calculates nDCG@5, AUC, Exposure L1, and Gini across mappings
bias/bootstrap.py	run_bootstrap(), percentile_ci()	In: predictions, 1000 samples Out: bootstrap_summary.json	Impression session resampling for 95% confidence intervals

Chapter 8: Experimental Setup, Capacity Probe & Benchmark Matrix

Hardware & Container Isolation Environment

All streaming benchmarks were executed in an isolated Linux environment using Docker Compose:

- **Resource Limits:** CPU / Memory Limits: Dedicated container CPU quotas (2.0 vCPU) and memory limits (4 GB RAM per engine) configured in `docker-compose.yml`.
- **Versions:** Software Stack: Python 3.11.17, PySpark 3.5.3, Apache Flink 1.20.1, Apache Kafka 3.3.0, Confluent-Kafka 2.6.1.

Single-Variable Capacity Probe (100 to 4,000 ev/s)

Before launching the 42-run matrix, we executed a capacity probe (`bench/probe.py`) across 6 rate tiers (100, 250, 500, 1000, 2000, 4000 ev/s) to locate the saturation ceiling:

- **Linear Regime:** Below 1,000 ev/s: All engines easily kept pace; output throughput matched input rate exactly.
- **Saturation Threshold:** At 2,000 ev/s and 4,000 ev/s: Single-node container processing capacity plateaued at ~1,730 events/second. Ingestion exceeding this ceiling caused persistent queue backlog in Kafka.

The 42-Run Benchmark Matrix Design

The matrix evaluates 7 configurations across 2 rates (1,000 ev/s below saturation, 2,000 ev/s above saturation) with 3 independent replications (60-second measurement windows following 30-second warmup):

Run Count: 7 Configs * 2 Rates * 3 Replications = Exactly 42 Benchmark Runs.

- **1.** spark-default: Spark with eager micro-batch trigger (lambda = 0.0)
- **2.** spark-1s: Spark with fixed 1-second micro-batch trigger (lambda = 0.0)
- **3.** flink-p1: Flink with parallelism=1 and default bundle buffer (lambda = 0.0)
- **4.** flink-p1-tuned: Flink with parallelism=1 and tuned 50ms bundle buffer (lambda = 0.0)
- **5.** flink-p2: Flink with parallelism=2 (lambda = 0.0)
- **6.** spark-default-rerank: Spark eager trigger with fair re-ranking enabled (lambda = 0.6)
- **7.** flink-p1-rerank: Flink tuned 50ms with fair re-ranking enabled (lambda = 0.6)

Chapter 9: Results, Analysis & Engineering Insights

Summary of Benchmark Matrix Measurements (Table 1)

Configuration	Rate (ev/s)	p50 Latency (ms)	p95 Latency (ms)	Throughput (ev/s)	Backlog (events)
spark-default	1000	292 +/- 2	529 +/- 31	997.5 +/- 2.1	0 +/- 0
spark-default	2000	5,410 +/- 320	11,850 +/- 840	1,735.2 +/- 12.4	18,420 +/- 510

spark-1s	1000	920 +/- 15	1,380 +/- 42	998.1 +/- 1.8	0 +/- 0
spark-1s	2000	5,840 +/- 410	12,410 +/- 910	1,728.4 +/- 15.1	18,910 +/- 620
flink-p1 (default)	1000	1,008 +/- 6	1,032 +/- 11	996.8 +/- 2.5	0 +/- 0
flink-p1-tuned (50ms)	1000	216 +/- 4	384 +/- 18	998.4 +/- 1.4	0 +/- 0
flink-p1-tuned	2000	5,120 +/- 290	11,240 +/- 780	1,741.0 +/- 14.2	17,950 +/- 480
flink-p2	1000	212 +/- 5	379 +/- 16	998.2 +/- 1.7	0 +/- 0
spark-default-rerank	1000	304 +/- 4	548 +/- 33	997.2 +/- 2.0	0 +/- 0
flink-p1-rerank	1000	224 +/- 5	396 +/- 21	998.0 +/- 1.6	0 +/- 0

Three Fundamental Systems Insights (Must Know for Viva)

- **1. Flink Buffer Tuning:** Insight 1 (The Flink Bundle Buffer Bottleneck): Out-of-the-box PyFlink buffers IPC output records to amortize Python-Java inter-process communication. Its default bundle time creates a 1,008 ms latency floor at 1,000 ev/s. By tuning `python.fn-execution.bundle.time = 50ms`, p50 latency plummeted from 1,008 ms to 216 ms, beating Spark (292 ms).
- **2. Spark Trigger Tradeoff:** Insight 2 (Spark Trigger Tradeoff): Spark Structured Streaming with an eager trigger achieved 292 ms latency. Introducing a 1-second fixed trigger increased latency to 920 ms, but reduced CPU utilization by 39% by consolidating micro-batch scheduling and driver coordination.
- **3. Reranker Cost:** Insight 3 (Fair Re-Ranking Overhead is Negligible): Comparing `spark-default` vs `spark-default-rerank` and `flink-p1-tuned` vs `flink-p1-rerank` at 1,000 ev/s shows that fair re-ranking added LESS THAN 1% CPU overhead and only 8-12 ms latency overhead. Real-time fairness is computationally cheap and production-viable.

Chapter 10: Limitations & Honest Disclosures

Examiners love students who honestly disclose project limitations rather than pretending their system is a flawless commercial deployment. Memorize these six limitations:

- **1. Publisher Labels:** 1. Synthetic Publisher Groups: MIND-small lacks real publisher identities. Our random and category-correlated mappings are analytical controls; our findings show the algorithm's mathematical behavior on real news text, but do not prove bias in Microsoft News.
- **2. Cost Estimates:** 2. Assumed Cloud Cost Model: All dollar cost figures ($0.0001/CPU-s, $0.00001/GB-s) are ASSUMED and illustrative based on standard cloud modeling, not live AWS/GCP bills.
- **3. State Persistence:** 3. Single-Node In-Memory Re-Ranker State: In our implementation, publisher exposure counts are kept in Python memory. If a node crashes, state resets. Production would use Redis or Flink Managed Keyed State.
- **4. Partition Divergence:** 4. Multi-Partition State Divergence: Across 2 Kafka partitions, each parallel worker tracks exposure independently. As proven in `tests/test_streaming.py`, partitioned local exposure totals sum to the global count, but can lead to slightly divergent re-ranking decisions.
- **5. Smoke Test Scope:** 5. Aggregate Offset Validation vs. Event Tracing: `make smoke` uses aggregate Kafka offset shell counts to verify message volume parity; it does not trace every individual event ID across the pipeline.
- **6. Smoke Test Logic:** 6. Pass-Through Mode in Smoke Test: `make smoke` runs `ENGINE_MODE=pass-through` to test Docker container transport in 30 seconds without loading heavy model tables. The full scoring algorithm is tested in unit tests and benchmark runs.

Chapter 11: Installation, Execution & Reproducibility Guide

Step-by-step verified terminal commands to reproduce the entire project from scratch:

1. Clone and Virtual Environment Setup:
```bash
git clone https://github.com/krishna28-sam/Recommender-at-Scale-Latency-Cost-and-Popularity-Bias.git
```

~/Big_Data
cd ~/Big_Data
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
export MIND_USER_HASH_SALT="local-secure-salt-2026"
```

2. Data Download and Feature Table Generation:
```bash
make data       # Fetches MIND-small from Azure Blob with Hugging Face fallback
make eda        # Validates dataset statistics -> data/processed/eda.json
make mappings   # Generates 6 publisher parquet/tsv mappings
make tables     # Builds Laplace CTR and category history tables
make hash-users # Generates salted SHA-256 hashed behaviors
```

3. Testing & Verification:
```bash
make test       # Runs 58 unit tests (56 passed, 2 skipped)
make smoke      # Starts Kafka, Spark, Flink, and tests offset parity
make summarize  # Re-aggregates 42 matrix runs -> matrix_summary.csv
make notebook   # Executes notebooks/results.ipynb (6 PNG charts)
make report     # Re-builds 4-page report.docx and report.pdf
```

Chapter 12: Team Presentation Strategy & Role Division

Suggested division of speaking responsibilities for our 10-12 minute classroom demonstration:

Presenter	Assigned Sections	Core Topics & Hand-off
Presenter 1: Krishna (PES1UG24AM285)	Introduction, Problem Statement, & Architecture (Slides 1-4)	Motivates popularity bias, explains dual-engine streaming architecture, explains Kafka setup and data flow. Hands off to Shree.
Presenter 2: Shree (PES1UG24AM265)	Dataset, Scoring Model & Fairness Re-Ranking (Slides 5-8)	Explains MIND-small, Laplace CTR formula, Exposure L1 distance, and Pareto frontier. Hands off to Shrivamshi.
Presenter 3: Shrivamshi (PES1UG24AM287)	Benchmark Matrix, Systems Results & Demo (Slides 9-12)	Presents Table 1, Flink bundle tuning insight, Spark trigger tradeoff, runs live `make smoke` demo, covers limitations and conclusion.

Chapter 13: Comprehensive Viva Voce Question Bank (30 Questions)

Here are 30 high-probability questions with detailed explanations and 15-second elevator pitches:

Q1: Why did you choose both Apache Spark and Apache Flink instead of just one?

Technical Explanation: Spark and Flink represent the two major paradigms in stream processing: micro-batching versus pipelined event-driven execution. Comparing them reveals fundamental trade-offs between latency, CPU efficiency, and tuning complexity.

> **[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)]** We chose both to compare micro-batching against event-driven streaming. Spark offers lower CPU cost via batching; Flink offers superior sub-second latency when tuned.
```

Q2: What is the difference between latency and throughput?

Technical Explanation: Latency is the turnaround time for a single event from arrival to output (in milliseconds). Throughput is the rate of processing events over time (events per second).

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Latency is response time per event; throughput is volume processed per second. A system can have low latency but saturate if throughput capacity is exceeded.

Q3: What happens when Kafka consumers cannot keep up with the producer?

Technical Explanation: Unconsumed messages accumulate in the Kafka topic log, causing consumer lag to increase. If sustained, queue backlog grows and end-to-end latency explodes.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Consumer lag and queue backlog swell. While Kafka safely buffers the data on disk, end-to-end latency spikes from milliseconds to tens of seconds.

Q4: What is backpressure?

Technical Explanation: Backpressure is a flow-control mechanism where a downstream bottleneck signals upstream producers to slow down transmission, preventing out-of-memory crashes.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Backpressure is flow control: downstream operators throttle upstream input to prevent system memory overload when ingestion exceeds capacity.

Q5: Why did you use MIND-small instead of MovieLens?

Technical Explanation: MIND is an authentic news recommendation dataset with article text, impressions, and click timestamps. MovieLens is a movie rating dataset lacking real news characteristics.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] MIND contains real news impression sessions and click logs. MovieLens was only prepared as a secondary backup pipeline.

Q6: Why are the publisher groups synthetic?

Technical Explanation: MIND-small does not contain a publisher tag. We synthesized 6 controlled publisher groups (3 random uniform, 3 category-correlated) to test exposure inequality rigorously.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] MIND lacks publisher metadata. We created 6 controlled synthetic mappings to test the mathematical behavior of our fair re-ranker on real news.

Q7: How does the re-ranker improve fairness?

Technical Explanation: It dynamically subtracts a penalty from candidate CTR scores proportional to the candidate publisher's current over-representation relative to its catalog target.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] It penalizes over-represented publishers based on cumulative exposure, allowing under-represented publishers to surface in top recommendations.

Q8: What does parameter lambda control?

Technical Explanation: Lambda is the fairness penalty weight. At lambda=0, recommendations are purely greedy CTR. Higher lambda forces stricter alignment with catalog publisher shares.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Lambda balances accuracy and fairness. At lambda=0, it maximizes CTR; higher lambda penalizes over-represented publishers to ensure fair exposure.

Q9: Why can fairness improve while nDCG decreases?

Technical Explanation: CTR-maximizing items are demoted in favor of fair publisher representation. Demoting a clicked article slightly lowers nDCG while boosting exposure fairness.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Promoting less-popular publishers for exposure fairness means occasionally demoting high-CTR articles, accepting a tiny utility drop for massive fairness gains.

Q10: Why is your cost analysis only illustrative?

Technical Explanation: We did not run on commercial cloud clusters with billed invoices. We modeled cost using standard cloud pricing rates applied to measured CPU-seconds and peak memory.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Cost figures are modeled using standard cloud prices ($0.0001/CPU-sec) applied to real container stats, not live billing invoices.

Q11: Does your streaming implementation guarantee exactly-once delivery?

Technical Explanation: No, it uses at-least-once delivery with Kafka. If a node terminates, uncommitted messages are replayed upon recovery.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] We use at-least-once processing. Transactions and offset commits ensure reliability, but transient duplicate replays can occur upon container crashes.

Q12: What does `make smoke` actually prove?

Technical Explanation: It proves multi-container orchestration, Kafka topic creation with LogAppendTime, salted SHA-256 user privacy hashing, and offset volume parity.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] It verifies infrastructure health: Kafka connectivity, Docker orchestration, user privacy hashing, and stream transport parity in 30 seconds.

Q13: What happens if you increase Kafka partitions?

Technical Explanation: Throughput capacity increases because consumers can scale horizontally. However, in-memory re-ranking state gets partitioned across workers.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] More partitions boost throughput via parallelism, but split stateful re-ranking counters across workers unless an external distributed cache is used.

Q14: Why did Flink's p50 latency drop from 1,008 ms to 216 ms?

Technical Explanation: PyFlink's Python worker buffers output records for up to 1,000 ms by default. Setting `python.fn-execution.bundle.time = 50ms` flushes records promptly.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] PyFlink defaults to a 1,000 ms bundle buffer. Tuning bundle time to 50 ms forces prompt flushing, slashing latency to 216 ms.

Q15: Why did Spark's 1-second trigger reduce CPU usage by 39%?

Technical Explanation: It consolidated micro-batch scheduling. Instead of launching dozens of tiny micro-batches per second, it executed one consolidated batch every second.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Batching once per second amortizes Spark driver scheduling and task coordination overhead, saving 39% CPU at the cost of higher latency.

Q16: What is the saturation throughput of your single-node system?

Technical Explanation: Approximately 1,730 events/second. Beyond this rate, input arrival exceeds engine processing speed and Kafka backlog swells past 18,000 events.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Saturation occurs at ~1,730 events/second on single-node containers. Beyond that, queue backlog grows and latency degrades severely.

Q17: What is Laplace smoothing and why alpha=10?

Technical Explanation: Laplace smoothing adds pseudo-counts (alpha=10) to raw CTR. It prevents brand-new articles with 1 impression and 1 click from having an inflated 100% CTR.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] It smooths click-through rates for new articles so that noise and cold-start items do not receive artificially dominant ranking scores.

Q18: How did you measure latency without clock synchronization issues?

Technical Explanation: Both input and output timestamps are assigned by the Kafka broker disk write (`LogAppendTime`). Measuring differences on the same broker eliminates clock skew.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] We used Kafka's `LogAppendTime`. Both timestamps are recorded by the same Kafka broker clock, completely eliminating clock synchronization error.

Q19: How did you validate that no user privacy is violated?

Technical Explanation: We verified that user IDs are salted and hashed with SHA-256 before publishing to Kafka. Raw user IDs never appear in stream payloads or logs.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] All user IDs are hashed with SHA-256 and an environment salt before entering Kafka. Zero raw user IDs exist in flight.

Q20: What is the difference between candidate nDCG@10 and served nDCG@5?

Technical Explanation: Candidate nDCG@10 measures the quality of the unconstrained candidate ranking. Served nDCG@5 measures the final top-5 recommendations served after re-ranking.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Candidate nDCG@10 evaluates the base recommender; served nDCG@5 evaluates the final top-5 list actually delivered after fair re-ranking.

Q21: Why did candidate nDCG@10 not change during re-ranking?

Technical Explanation: Re-ranking selects which items to serve; it does not alter the underlying candidate scoring function.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Re-ranking changes the top-5 served list, not the candidate generation model or candidate score distribution.

Q22: What is the 1,000-resample bootstrap?

Technical Explanation: A non-parametric statistical technique that repeatedly resamples 73,152 validation impressions with replacement 1,000 times to compute 95% confidence intervals.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] It resamples validation impressions 1,000 times to prove our observed fairness gains are statistically significant and not random chance.

Q23: Did the 95% confidence intervals for fairness gains exclude zero?

Technical Explanation: Yes. For all six publisher mappings, the 95% bootstrap confidence intervals for Exposure L1 reduction strictly excluded zero (39.98% to 43.89% reduction).

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Yes! The 95% confidence intervals for Exposure L1 reduction strictly exclude zero across all six mappings, proving statistical significance.

Q24: What is the computational overhead of the fair re-ranker in streaming?

Technical Explanation: Less than 1% CPU utilization and 8-12 ms latency overhead below saturation.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Fair re-ranking adds less than 1% CPU and under 12 ms latency, making real-time fairness highly practical.

Q25: How does your system handle cold-start items?

Technical Explanation: Cold-start items receive a smoothed CTR based on global category priors and Laplace smoothing constant alpha=10.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Cold-start items fall back to category-level CTR priors via Laplace smoothing, preventing them from being buried or over-promoted.

Q26: Why did you use Docker Compose?

Technical Explanation: Docker Compose provides automated, reproducible multi-service orchestration for Kafka, Spark, Flink, and the replay producer.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] It ensures total reproducibility, spinning up Kafka, Spark, and Flink with exact networking and resource limits on any Linux host.

Q27: How would you scale this system to 100,000 events/second in production?

Technical Explanation: Scale Kafka partitions to 32+, deploy Spark/Flink on a Kubernetes cluster with multiple worker nodes, and externalize state to Redis.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Scale Kafka partitions, deploy Flink on Kubernetes worker pods, and store re-ranking state in a distributed Redis cache.

Q28: What is the Gini coefficient of publisher exposure?

Technical Explanation: It measures inequality in recommendation impressions across publishers from 0.0 (perfect equality) to 1.0 (monopoly).

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] The Gini coefficient quantifies exposure inequality across publishers on a scale from 0 to 1.

Q29: Why does Flink require a Java connector JAR?

Technical Explanation: PyFlink runs Python logic via Py4J, but relies on Apache Flink's optimized Java streaming runtime and Kafka SQL connectors.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] PyFlink is a Python API over Flink's high-performance Java streaming core, requiring the Kafka SQL connector JAR at runtime.

Q30: What is your project's single most important takeaway?

Technical Explanation: Real-time fair re-ranking is computationally viable in high-throughput streaming, and engine buffer configuration matters more than raw engine choice.

[WHAT TO SAY IN 15 SECONDS (ELEVATOR PITCH)] Fairness mitigation is computationally inexpensive, and tuning streaming buffer parameters is critical to achieving sub-second latency.

Chapter 14: Quick Revision Sheets & Numerical Cheat Sheet

Keep this page in front of you 10 minutes before your viva starts.

Key Numbers to Memorize

- **Validation Impressions:** 73,152 impressions (MIND-small validation set)
- **News Catalog Size:** 51,282 articles in train; 42,416 articles in dev; 65,234 unique articles total
- **Benchmark Matrix Runs:** 42 runs (7 configs * 2 rates * 3 replications, 60s measurement windows)
- **Saturation Threshold:** ~1,730 events/second (beyond which queue backlog swells past 18,000 events)
- **Flink Tuned p50 Latency (1k ev/s):** 216 ms (beating Spark's 292 ms and default Flink's 1,008 ms)
- **Spark 1s Trigger Effect:** 39% CPU savings at the cost of increasing p50 latency from 292 ms to 920 ms
- **Exposure L1 Reduction:** 40.0% to 43.9% reduction across all 6 publisher mappings at lambda=5.0
- **Utility Trade-off:** Only 1.18% to 4.35% drop in served nDCG@5 at lambda=5.0
- **Reranker Streaming Overhead:** <1% CPU increase and 8-12 ms latency increase at lambda=0.6
- **Bootstrap Resamples:** 1,000 resamples of 73,152 sessions (seed 20261028)

Architecture & Data Flow in 4 Steps

- 1. Producer hashes user IDs with salted SHA-256 and streams to Kafka `impressions` topic.
- 2. Kafka timestamps each message on disk using `LogAppendTime` (eliminating clock skew).
- 3. Spark/Flink scores candidates via Laplace CTR (alpha=10) and applies greedy fair re-ranking penalty.
- 4. Recommendations emitted to `recommendations` topic; monitor measures latency, throughput, backlog, and cgroup CPU.